FORTH

28 avril 2020

Damien Morard

Final date : 25 mai 2020 à 23h59

Files to include : Si vous choisissez Logickit, vous mettrez vos fichiers dans le dossier correspondant (de même pour Prolog). Vous fournirez un rapport et le code correspondant.

Les TPs sont des travaux *personnels*, si deux solutions ont des similarités flagrantes les deux personnes auront 0.

La date et l'heure de rendu sont *strictes*, passer le délai d'une minute utilisera un joker (pour une journée supplémentaire). Une fois les deux jokers du semestres consommés et le délai dépassé, vous recevrez une note de 0. Bien entendu la date et l'heure de rendu sont toujours considérées au fuseau horaire de Genève.

Vous devez rendre votre travail sous format pdf !

Ce TP présente un langage assez complexe, avec la notion d'appel de fonction, de saut et de condition.

Les Jedi ne font rien comme tout le monde. Pour programmer, ils utilisent un langage assez spécial : FORTH [1]. Celui-ci a la particularité d'être un langage à pile, c'est-à-dire que ses données sont empilées et les instructions manipulent uniquement la pile Les opérations consomment les éléments du haut de la pile, et leurs résultats sont empilés. Tout au long de ce TP, nous écrirons la pile sous la forme d'une liste, par exemple `[42,51,8]`, où `42` est l'élément au sommet de la pile.

Par exemple, le programme suivant duplique l'élément en haut de la pile avec `DUP` puis applique l'opérateur `*`. Ceci calcule le carré du nombre en haut de la pile.

1. http://fr.wikipedia.org/wiki/forth_(langage)

```
11 DUP *   ( on obtient 121 en sommet de la pile )
```

La première instruction (`11`) pousse `11` au sommet de la pile. La seconde instruction (`DUP`) duplique l'élément au sommet de la pile. Celle-ci devient donc `[11,11]`. La troisième instruction multiplie les deux éléments au sommet de la pile (et les consomme), puis place le résultat sur la pile. Celle-ci vaut donc `[121]` à la fin de l'exécution.

Les seuls opérateurs que nous considérons sont décrits ci-dessous.

```
<number>   ( a number is pushed on the stack )
+, -, *, / ( two numbers are consumed from the stack
             the result is pushed on the stack )
LABEL=<n>  ( defines label n )
JUMP=<n>   ( jumps to label n
             if the top of the stack is equal to 0 )
SWAP       ( Reverses the top two stack items)
DUP        ( Duplicates the top stack item)
OVER       ( Makes a copy of the second item
             and pushes it on the stack.)
DROP       ( Discards the top stack item.)
```

Les instructions `LABEL` et `JUMP` sont un ajout par rapport à `FORTH`. Elles permettent de sauter à un endroit du programme si une condition est vérifiée. Ici, la condition est que l'élément en haut de la pile soit 0. Dans le cas contraire, l'exécution continue avec l'instruction suivante.

Exercise 1 : Manipulation de pile

`FORTH` utilise la notion de pile. Vous commencerez donc par définir une pile. Celle-ci dispose des opérations suivantes :

- `empty` qui correspond à une pile vide ;
- `push(p, e)` qui retourne une pile dont le premier élément est `e` et est suivi de la pile `p` ;
- `top(p)` qui retourne le premier élément de la pile `p` ;
- `pop(p)` qui retourne la pile `p` privée de son premier élément.

1. Définissez, sous forme de règles d'inférence, la syntaxe abstraite du langage de manipulation de piles décrit ci-dessus.
2. Donnez la sémantique d'évaluation $\overset{stack}{\Longrightarrow}$ de ces opérations sous la forme de règles d'inférence, en précisant bien le domaine sémantique.
3. Écrivez en LogicKit ou Prolog les règles permettant de manipuler une pile.

Exercise 2 : Avec des GOTOs...

1. Quel est le résultat du programme suivant ? Détaillez les étapes d'exécution de manière informelle.

```
1 1 2 + + LABEL=1 4 - JUMP=1 5 +
```

2. Donnez la syntaxe concrète de ce langage en notation EBNF.
3. Donnez la syntaxe abstraite de ce langage sous formel de règles d'inférence.
4. Donnez la sémantique d'évaluation $\overset{Ex2}{\Longrightarrow}$ d'un programme `FORTH` avec les instructions `JUMP` et `LABEL`.
5. Écrivez en LogicKit ou Prolog les règles corresponantes.

Exercise 3 : Et avec des fonctions ?

Pour cet exercice, nous oublions les instruction `LABEL` et `JUMP`. La déclaration d'une fonction se fait de la manière suivante :

```
: CARRE DUP * ; ( definition de CARRE )
```

Remarquez que la définition est entourée par les caractères `:` et `;`. C'est une instruction comme une autre, qui déclare une fonction.

1. Évaluez le programme suivant de manière informelle :

```
: CARRE DUP * ;  ( definition de CARRE )
11 CARRE         ( on obtient 121 en sommet de la pile )
```

2. Donnez la syntaxe concrète de ce langage en notation EBNF.
3. Donnez la syntaxe abstraite de ce langage sous formel de règles d'inférence.
4. Donnez la sémantique d'évaluation $\overset{Ex3}{\Longrightarrow}$ d'un programme `FORTH` avec fonctions.
5. Écrivez en LogicKit ou Prolog les règles corresponantes.
6. Est-il possible d'avoir des fonctions récursives ? Que peut-il se passer ?

Exercise 4 : Conditionnelles

Pour cet exercice, nous oublions les instruction `LABEL` et `JUMP`, mais pas les fonctions. La fonction suivante [2] affiche le calibre d'un œuf à partir de sa taille.

```
: EGGSIZE  DUP  18 < IF ." reject "       ELSE
           DUP  21 < IF ." small "        ELSE
           DUP  24 < IF ." medium "       ELSE
           DUP  27 < IF ." large "        ELSE
           DUP  30 < IF ." extra large "  ELSE
                       ." error "
           THEN THEN THEN THEN THEN DROP ;
```

La condition est située avant l'instruction `IF`. Ainsi, `0 1 =` empile `faux` alors que `0 1 <` empile `vrai`. Si la condition est respectée, les instructions suivant le `IF` sont exécutées, sinon les instructions suivant le `ELSE` correspondant sont exécutées.

1. Quelle différence y a-t-il par rapport au `GOTO` vu précédemment ?
2. Donnez la syntaxe concrète de ce langage en notation EBNF.
3. Donnez la syntaxe abstraite de ce langage sous formel de règles d'inférence.
4. Donnez la sémantique d'évaluation $\overset{Ex4}{\Longrightarrow}$ d'un programme `FORTH` avec conditions.
5. Écrivez en LogicKit ou Prolog les règles corresponantes.

Que le FORTH soit avec vous !

2. Prise sur http://www.forth.com/starting-forth/sf4/sf4.html